

Section 1 - Design

Q1. What do you consider a 'state' in this environment? How would you define a 'neighboring state' for this problem?

I will consider a state in this environment to be some board state at any point, a neighboring state would be one with one more or fewer shapes

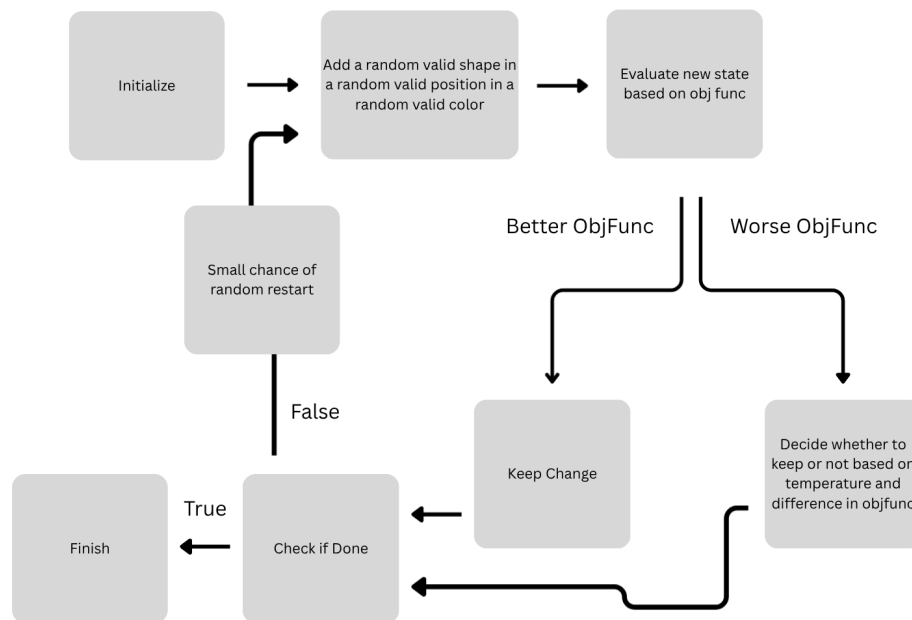
Q2. First-choice local search requires that you move to a random neighboring state. How do you (plan to) implement random neighbor selection? Explain in 1-2 sentences.

A random neighbor selection would be done by randomly adding a shape, the position and color of the added shape must be valid

Q3. Specify your objective function. Are all constraints modeled through the objective function, or are some treated as explicit rules? Explain your choices.

Because the goal is to use 1. as few colors, 2. as few shapes, and 3. fill the board. my objective function will be a product of a measure of the 3. I will use variable constants a, b, and c in the objective function: $a(\# \text{ of colors}) + b(\# \text{ of shapes}) + c(\# \text{ of empty cells})$ with a goal is to minimize.

Q4. Draw an end-to-end overview diagram/flowchart for your (proposed) local search approach. If using any extensions to local search (such as random bad moves or restarts), that workflow must be reflected in your flowchart.



Section 3 - Analysis/Reflections

Q1. Did you encounter local optima when testing your implementation? If so, when did this happen (structurally, what was going on with the state)? If not, provide an example of a local optimum that could get the agent stuck, where the only way out would be to backtrack to a previous state.

Because I used aspects of simulated annealing, my implementation was less susceptible to getting stuck with local optima, although I did witness times where it would get stuck with a couple of empty spaces and exhaust the iteration limit I put in place due to the random nature of local search.

Q2. Is your solution optimal or near-optimal? Explain why/why not.

Because I incorporated simulated annealing design choices, my solution should be more optimal than purely using local search.

AI Use disclosure:

I used Claude code to review my code for bugs and to find methods to favor smaller shapes when the solution was almost complete, `find_valid_color` in `hw1.py` was written by Claude which was an expansion on `getAvailableColor` in `gridgame.py`.